

THE TASK:

Project: RoboDog – GPS + Local Navigation

Objective: Enable the robot dog to navigate autonomously to a target GPS coordinate using ROS2.

Requirements:

- Add a GPS source (hardware or external device)
- Publish GPS data (latitude, longitude, status) to a ROS2 topic
- Robot receives a target (lat/lon) via ROS2
- Robot must:
 - Move in the correct direction
 - Avoid obstacles (simple local behavior)
 - Stop near the target

Important Notes:

- No map required
- No SLAM required
- GPS does NOT need to be directly attached to the robot
- Focus on a simple working system

THE ALGORITHM (CORE LOGIC):

The navigation logic is the same for all options.

Step 1 - Get Position: Read current GPS position (lat, lon)

Step 2 - Get Target: Receive target coordinates via ROS2

Step 3 - Compute Distance: Calculate distance between current position and target

Step 4 - Compute Direction (Bearing): Determine direction from current position to target

Step 5 - Movement Decision: If aligned → move forward, **and** If angle is off → turn left/right

Step 6 - Obstacle Avoidance: If obstacle detected:

- turn left or right
- move around
- continue toward target

Step 7 - Stop Condition

- If distance < threshold (e.g. 1–2 meters) **then** STOP

GPS OPTIONS (SHORT OVERVIEW)

We have 4 possible GPS integration options:

1. Smartphone (wireless GPS)
2. USB GPS Module + PC
3. ESP/Arduino + GPS Module
4. Onboard Robot Computer (Raspberry Pi inside robot)

OPTION 1: Smartphone-Based GPS Integration (Onboard Access Required)

Description: A mobile phone is used as an external GPS sensor. The device acquires positioning data and transmits it wirelessly (e.g., via WiFi) to the robot's onboard computing unit, where it is processed within the ROS2 framework.

System Architecture: Phone → Wireless Communication → Onboard Computer (ROS2) → Robot

Advantages: No additional hardware is required, rapid implementation and deployment, utilizes reliable, built-in GPS capabilities.

Disadvantages: Requires stable wireless communication setup, the phone should be physically mounted on the robot to ensure positional accuracy, dependent on external device and network reliability.

OPTION 2: External USB GPS Module with PC (No Onboard Access Required)

Description: A dedicated GPS module (e.g., u-blox) is connected to an external computer via USB. The computer runs ROS2, processes the GPS data, and communicates control commands to the robot.

System Architecture: GPS Module → USB → External PC (ROS2) → Robot

Advantages: Clear and modular system architecture, direct compatibility with ROS2, independent of wireless communication.

Disadvantages: Requires parsing of raw GPS data (e.g., NMEA format), requires external computing hardware, reduced system integration compared to onboard solutions.

OPTION 3: Microcontroller-Based GPS Integration (Onboard Access Required)

Description: A GPS module is connected to a microcontroller (e.g., ESP32 or Arduino), which reads and transmits the positioning data to the robot's onboard computer. The onboard system then publishes the data to ROS2.

System Architecture: GPS Module → Microcontroller → Onboard Computer (ROS2) → Robot

Advantages: Provides low-level hardware control, suitable for embedded system experimentation, flexible interfacing with various sensors.

Disadvantages: Microcontrollers cannot run ROS2 natively, requires an additional communication layer between microcontroller and onboard system, increased system complexity and debugging effort.

OPTION 4: Direct GPS Integration with Onboard Computer (Onboard Access Required)

Description: The GPS module is directly connected to the robot's onboard computing unit (e.g., Raspberry Pi). The onboard system reads, processes, and publishes GPS data within ROS2 without intermediate devices.

System Architecture: GPS Module → Onboard Computer (ROS2) → Robot

Advantages: Fully integrated and self-contained system, eliminates need for external devices, direct and efficient ROS2 communication.

Disadvantages: Requires full access to the robot's internal system, potential risk of affecting the robot's existing configuration, limited debugging flexibility compared to external setups.

HOW TO DECIDE

We choose based on:

1. **Implementation Speed:** How fast can we build and test?
2. **Complexity:** How many components and layers?
3. **Reliability:** Stable communication and data flow.
4. **Risk:** Risk of breaking robot or wasting time.
5. **Project Scope:** Keep solution simple (no overengineering).

SUMMARY TABLE

Option	Onboard Access Required	Complexity	Implementation Speed	Reliability	ROS2 Integration	Key Limitations
Smartphone GPS	Yes	Low	Very High	Medium	Indirect (via onboard system)	Depends on wireless communication and external device
USB GPS Module (External PC)	No	Medium	High	High	Direct	Requires external computer and GPS data parsing
Microcontroller (ESP/Arduino)	Yes	High	Low	Medium	Indirect (via onboard system)	Cannot run ROS2, adds extra communication layer
Direct Onboard GPS Integration	Yes	Medium	Medium	High	Direct	Requires full system access and careful integration